

Jour 2 – Data manipulation

Agrégation, jointures, dates et texte – objectif 15 minutes par notebook

1. Objectif de la journée et périmètre

Cette journée couvre **10** briques : `groupby`, `agg`, `merge`, `join`, `concat`, `pd.to_datetime/ .dt`, `.str`, `apply`, `map`, `np.where`. On suppose les fondamentaux du Jour 1 (`loc/iloc`, `isna/fillna`, `value_counts`, etc.) déjà acquis – ils sont réutilisés sans être ré-expliqués.

Pourquoi ce découpage colle au format BCG X. Une fois les données inspectées et nettoyées (Jour 1), la question BCG X passe presque toujours à l'étape suivante : *résumer par groupe* (combien, en moyenne, par ville/segment/période), *combinaison plusieurs sources* (une table de courses, une table de chauffeurs, une table de tarifs) et *extraire une information temporelle ou textuelle* (le mois, le jour de semaine, un mot-clé dans un commentaire). Ce sont exactement les trois familles couvertes ici.

2. Comment utiliser ce support

1. Faites d'abord le notebook `notebook/day2_data_manipulation_exercices.ipynb` seul, chrono à 15 minutes, sans ouvrir ce PDF ni les solutions.
2. Corrigez avec `solutions/day2_data_manipulation_solutions.ipynb`.
3. Revenez ici pour la partie « **comment raisonner** » de chaque famille de question.
4. Refaites le notebook le lendemain à froid, avant de passer au Jour 3.

3. Dates – `pd.to_datetime`, l'accessor `.dt`

```
df["d"] = pd.to_datetime(df["d"]) # texte -> vrai type date, TOUJOURS en 1er
df["d"].dt.year / .dt.month / .dt.day
df["d"].dt.day_name() # "Monday", ...
df["d"].dt.weekday # 0=lundi (entier)
df["d"].dt.is_month_end
```

COMMENT RAISONNER FACE À CETTE QUESTION

Un CSV ne contient que du texte : tant que `pd.to_datetime` n'a pas été appelé explicitement, une colonne de dates reste un **object** et `.dt` n'existe pas. Sans conversion, trier cette colonne trie par ordre **alphabétique** de la chaîne – ça fonctionne par coïncidence au format `AAAA-MM-JJ`, mais casse totalement sur un format `JJ/MM/AAAA`.

Une fois convertie, tous les attributs temporels passent par l'accessor `.dt` – exactement le même principe que `.str` pour le texte (section 5) : un accesssor dédié au type réel de la colonne, plutôt que des fonctions éparpillées.

SCHÉMA À RECONNAÎTRE

« colonne de date issue d'un CSV » → `pd.to_datetime` en tout premier, avant le moindre tri, filtre ou calcul temporel.

ERREURS FRÉQUENTES

- Oublier de réaffecter (`df["d"] = pd.to_datetime(df["d"])`) – `pd.to_datetime` ne modifie jamais en place.
- Appeler `.dt` sur une colonne encore en `object` (non convertie) → `AttributeError`.

4 4. Groupement – groupby, agg

```
df.groupby("ville")["prix"].mean() # une metrique, une cle
df.groupby("ville").size() # nombre de LIGNES par groupe
df.groupby("ville")["prix"].count() # nombre de non-NaN par groupe

df.groupby("ville").agg(
    total=("prix", "sum"),
    moyenne=("prix", "mean"),
    n=("id", "count"),
) # plusieurs metriques, noms propres

df.groupby(["ville", "statut"])[ "prix"].mean() # cle composite -> MultiIndex
df.groupby(["ville", "statut"])[ "prix"].mean().unstack() # -> tableau large
```

COMMENT RAISONNER FACE À CETTE QUESTION

`groupby("cle")` seul ne calcule rien : c'est un objet « en attente ». Il faut toujours préciser le triplet complet : **quelle clé** de regroupement, **quelle colonne** agréger, **quelle fonction** appliquer.

size() contre count() – ne pas confondre. `size()` compte **toutes** les lignes du groupe, même celles avec des NaN sur la colonne d'intérêt. `count()` (appelé sur une colonne précise) ignore ses NaN. Pour répondre à « combien de lignes/courses/clients par groupe », c'est presque toujours `size()` qu'il faut.

La named aggregation (`agg(nom=("col", "fn"))`) est la forme à privilégier dès que plusieurs statistiques sont demandées : une seule passe sur les données, des noms de colonnes de sortie choisis explicitement – pas de `MultiIndex` de colonnes à aplatir ensuite comme avec l'ancienne syntaxe `agg({"col": ["sum", "mean"]})`.

Grouper par plusieurs clés produit un résultat indexé par le *couple* de valeurs – exploitable tel quel, mais souvent plus lisible après un `.unstack()` qui fait passer le dernier niveau de l'index en colonnes (format « large », proche d'un tableau croisé).

SCHEMA À RECONNAÎTRE

« moyenne/somme/compte par catégorie » → `groupby("cle")["valeur"].fonction()`. « plusieurs statistiques nommées » → `agg(nom=("col", "fn"))`. « croiser deux catégories » → `groupby([c1, c2])` puis `.unstack()`.

ERREURS FRÉQUENTES

- Utiliser `count()` en pensant « nombre de lignes » alors que la colonne choisie contient des NaN – le résultat est alors inférieur au nombre réel de lignes, silencieusement.
- Oublier que `groupby` trie par défaut les clés de groupe (paramètre `sort=True`) – l'ordre du résultat n'est pas forcément l'ordre d'apparition dans les données d'origine.

5 5. Texte – l’accessor `.str`

```
df["v"].str.strip() # espaces en debut/fin
df["v"].str.lower() / .str.upper() / .str.title()
df["v"].str.contains("mot", case=False, na=False) # sous-chaine, insensible casse
df["v"].str.replace("a", "b")
df["v"].str.split(",")
df["v"].str.len()
```

COMMENT RAISONNER FACE À CETTE QUESTION

Comme `.dt` pour les dates, `.str` est l’accessor qui vectorise les méthodes de chaînes Python sur toute une colonne – `strip()`, `title()`, etc. s’enchaînent exactement comme sur un `str` normal, mais s’appliquent ligne par ligne à toute la Series.

`na=False` dans `.str.contains` n’est pas optionnel en pratique. Sans ce paramètre, une ligne où la colonne testée est manquante produit `NaN` (pas `False`) – un masque booléen contenant des `NaN` lève une erreur ou se comporte de façon imprévisible dès qu’on l’utilise pour filtrer (`df[masque]`). `na=False` force ces cas à `False`, presque toujours le comportement voulu.

Toujours normaliser avant de comparer ou de joindre. Deux chaînes qui représentent la même valeur métier ("`paris`", "`PARIS`", "`Paris`") sont trois valeurs **différentes** pour Python tant qu’elles n’ont pas été nettoyées. Le réflexe systématique avant toute comparaison, `isin`, ou clé de `merge` sur du texte : `.str.strip()` puis une casse homogène (`.str.lower()` ou `.str.title()`).

SCHÉMA À RECONNAÎTRE

« *texte à comparer ou à joindre* » → `strip()` + casse homogène, systématiquement, avant la comparaison. « *chercher un mot-clé, colonne avec des manquants* » → `.str.contains(..., na=False)`.

ERREURS FRÉQUENTES

- Comparer une colonne déjà nettoyée à une colonne encore brute – le résultat est faux **sans lever d’erreur**, le piège le plus sournois de cette famille.
- Oublier `na=False` sur `.str.contains` puis s’étonner que le masque booléen ne filtre pas comme prévu sur les lignes à valeur manquante.

6 6. Combiner des tables – merge, join, concat

```
df_a.merge(df_b, on="cle", how="inner") # ne garde que les cles des DEUX cotes
df_a.merge(df_b, on="cle", how="left") # garde TOUTES les lignes de df_a
df_a.merge(df_b, on="cle", how="left", suffixes=("_a", "_b"))

df_a.set_index("cle").join(df_b.set_index("cle")[["col"]]) # jointure sur INDEX

pd.concat([df1, df2], ignore_index=True) # empiler des LIGNES, meme schema
pd.concat([df1, df2], keys=["lot1", "lot2"]) # + trace du lot d'origine
```

COMMENT RAISONNER FACE À CETTE QUESTION

Trois familles à ne jamais confondre. `concat` empile des **lignes** (même schéma de colonnes, plusieurs lots à rassembler). `merge/join` ajoutent des **colonnes** à partir d'une clé commune (enrichir une table à l'aide d'une autre). Confondre les deux familles est la source numéro un d'erreurs sur ce type d'exercice – avant d'écrire du code, demandez-vous toujours : « est-ce que j'empile des lignes, ou est-ce que j'enrichis des colonnes ? »

`how="inner"` contre `how="left"`. `inner` ne garde que les clés présentes des deux côtés – des lignes disparaissent silencieusement si une clé manque d'un côté. `left` garde **toutes** les lignes de la table de gauche (la table de référence), avec des `NaN` là où aucune correspondance n'existe. Le choix dépend d'une question simple : « est-ce que je veux analyser uniquement les cas où tout est connu, ou absolument toutes les lignes de ma table principale ? »

Vérifier l'unicité de la clé côté table de dimension avant tout `merge`. Si la table de droite (souvent une table de référence comme « chauffeurs » ou « produits ») contient des clés **dupliquées**, chaque ligne de la table de gauche qui matche cette clé est **multipliée** autant de fois qu'il y a de doublons côté droit – le nombre de lignes après `merge` explose au lieu de baisser, souvent sans qu'aucune erreur ne soit levée. Le réflexe de sécurité : `assert df_b["cle"].is_unique` avant de merger, et comparer systématiquement `len()` avant/après. **`.join()` est un raccourci de `merge` sur l'index.** Il suppose que les deux DataFrames sont déjà indexés par la clé commune (typiquement après un `set_index` ou un `groupby`, dont le résultat est indexé par la clé de groupe). Dès que la clé est une colonne normale, ou que les deux tables n'ont pas le même index, `merge` reste le choix par défaut, plus explicite.

`concat` empile selon les labels de colonnes communs. Si deux DataFrames n'ont pas exactement les mêmes colonnes, des `NaN` apparaissent là où une colonne manque d'un côté – comportement normal, à anticiper plutôt qu'à subir.

SCHEMA À RECONNAÎTRE

« je veux garder toutes les lignes de ma table principale » → `merge(..., how="left")` avec la table principale à gauche. « je ne veux que les correspondances valides des deux côtés » → `how="inner"`. « les deux tables sont déjà indexées par la même clé » → `.join()`. « recoller des lots de même schéma » → `pd.concat(..., ignore_index=True)`.

ERREURS FRÉQUENTES

- Ne pas vérifier `len()` avant/après un `merge` – c'est la seule façon fiable de détecter une explosion de lignes causée par une clé dupliquée côté droit.
- Confondre « aucune ligne perdue » (`how="left"`) avec « aucune donnée manquante » – un `left join` garde toutes les lignes mais peut introduire de nouveaux `NaN`.
- Oublier `ignore_index=True` dans un `concat` de deux DataFrames issus de la même source – leurs index se chevauchent, et un `.loc[3]` peut soudain renvoyer plusieurs lignes au lieu d'une.
- Appeler `.join()` sans avoir indexé les deux DataFrames par la même clé – échoue ou joint silencieusement sur l'index numérique par défaut, sans rapport avec la clé métier voulue.

7 7. Transformer – `apply`, `map`, `np.where`

```
df["v"].apply(fonction) # Series -> Series, logique arbitraire (if/elif...)
df.apply(fonction, axis=1) # une LIGNE entière -> une valeur (multi-colonnes)
df["v"].map({"a": 1, "b": 2}) # correspondance SIMPLE valeur -> valeur (dict)
np.where(condition, si_vrai, si_faux) # if/else vectorise, sans fonction
np.where(cond1, v1, np.where(cond2, v2, v3)) # >2 branches, par imbrication
```

COMMENT RAISONNER FACE À CETTE QUESTION

Trois outils, trois usages, ne pas les substituer au hasard. `map` sert uniquement à une correspondance simple valeur-par-valeur via un dictionnaire (ou une `Series`) – l'équivalent d'un « RECHERCHEV » sur une seule colonne, sans aucune condition possible. `apply` sert dès qu'une véritable logique (plusieurs branches, plusieurs colonnes en entrée) est nécessaire – `Series.apply` pour une colonne, `DataFrame.apply(axis=1)` quand le calcul combine plusieurs colonnes de la même ligne. `np.where` sert à un `if/else` **vectorisé** sur une condition simple, sans écrire de fonction – nettement plus rapide qu'un `apply` équivalent, et s'imbrique pour gérer plus de deux branches (au-delà de 2-3 niveaux, revenir à une fonction + `apply`, ou `pd.cut` pour des tranches numériques, devient plus lisible).

Les clés absentes d'un `map` deviennent NaN silencieusement. Si une valeur de la colonne source n'a pas d'entrée dans le dictionnaire, `map` ne lève pas d'erreur – elle produit un NaN à cet endroit. Vérifier `.isna().sum()` juste après un `map` en contexte métier réel est un réflexe, pas une option : une grille de correspondance incomplète est un bug silencieux classique. À l'inverse, `replace` ne remplace que les valeurs listées explicitement et laisse les autres inchangées – à choisir quand le besoin est « remplacer certaines valeurs, garder les autres telles quelles ».

`np.where` sur une condition impliquant un NaN. Une comparaison numérique avec un NaN (`NaN > 15`) est toujours `False` en pandas/numpy – `np.where` choisit donc la branche « faux » pour ces lignes, jamais un NaN en sortie. C'est parfois le comportement voulu, mais jamais à supposer sans vérifier : si les manquants doivent être traités à part, ajouter un test explicite `pd.isna(...)` en amont de la logique.

Toujours gérer les cas limites dans la fonction passée à `apply`. Une division par une colonne pouvant valoir 0 ou NaN doit être testée explicitement (`pd.isna(x) or x == 0`) avant de diviser – sinon le résultat est `inf` (pas une erreur, pas un NaN), une valeur qui passe inaperçue dans un `.isna().sum()` mais fausse silencieusement toute moyenne ou tri calculé ensuite sur la colonne.

RACCOURCI PLUS RAPIDE

`np.where` bat toujours un `.apply()` équivalent dès que la règle tient en une ou deux conditions simples sur une colonne entière – réserver `apply` aux logiques que `np.where` ne peut pas exprimer proprement.

ERREURS FRÉQUENTES

- Utiliser `map` en pensant qu'elle laisse les valeurs non trouvées **inchangées** comme `replace` – non, elle les remplace toutes par NaN.
- Diviser deux colonnes sans gérer le dénominateur nul/manquant – produit des `inf` invisibles dans un contrôle NaN.
- Utiliser `df.apply(axis=1)` par réflexe là où une formulation vectorisée (`np.where`, opérations directes sur les colonnes) existerait – correct mais inutilement lent sur de gros volumes.

8 8. Méthode générale face à une question BCG X

Avant d'écrire la moindre ligne de code, identifiez la **famille** de la question :

L'énoncé demande...	Famille → réflexe
« moyenne/somme/compte par catégorie »	<code>groupby("cle")["v"].fn()</code>
« plusieurs statistiques, noms propres »	<code>groupby(...).agg(nom=(...))</code>
« croiser deux catégories »	<code>groupby([c1,c2])[...].unstack()</code>
« convertir/extraire une date »	<code>pd.to_datetime</code> puis <code>.dt</code>
« nettoyer/chercher dans du texte »	<code>.str.strip/.lower/.contains</code>
« enrichir une table à partir d'une clé »	<code>merge(..., on=, how=)</code>
« tables déjà indexées par la même clé »	<code>.join()</code>
« recoller des lots de même schéma »	<code>pd.concat(..., ignore_index=True)</code>
« correspondance simple valeur→valeur »	<code>.map(dict)</code>
« logique arbitraire, plusieurs branches »	fonction + <code>.apply(...)</code>
« if/else simple, vectorisé »	<code>np.where(cond, v1, v2)</code>

COMMENT RAISONNER FACE À CETTE QUESTION

Comme au Jour 1 : l'erreur coûteuse en épreuve chronométrée n'est pas de connaître la syntaxe, c'est de passer du temps à redécouvrir quelle famille d'outil s'applique. Ici, la confusion la plus fréquente oppose `concat` (empiler des lignes) à `merge/join` (enrichir des colonnes) – si un doute subsiste en épreuve, revenir d'abord à cette question avant d'écrire quoi que ce soit.

9. Check-list finale

- ☐ `pd.to_datetime` appliqué avant tout calcul sur une colonne de date
- ☐ `.dt` pour les dates, `.str` pour le texte – jamais de boucle Python manuelle
- ☐ texte nettoyé (`strip` + casse homogène) avant toute comparaison ou jointure
- ☐ `.str.contains(..., na=False)` dès que la colonne testée peut contenir des manquants
- ☐ `groupby(...)[...].fonction()` avec le triplet clé/colonne/fonction toujours explicite
- ☐ `size()` vs `count()` choisi consciemment selon la présence de NaN
- ☐ *named aggregation* (`agg(nom=(...))`) dès que plusieurs métriques sont demandées
- ☐ `merge how= (inner/left)` choisi selon le besoin, `len()` vérifié avant/après
- ☐ unicité de la clé côté table de dimension vérifiée avant un `merge`
- ☐ `.join()` réservé aux tables déjà indexées par la même clé
- ☐ `concat` avec `ignore_index=True` ou `keys=` selon le besoin de traçabilité
- ☐ choix conscient entre `map` (correspondance simple), `apply` (logique arbitraire) et `np.where` (if/else vectorisé)
- ☐ division/ratio protégés contre le dénominateur nul ou manquant
- ☐ le notebook `day2_data_manipulation_exercices.ipynb` bouclé en moins de 15 minutes